

## Sistema de Vendas de Passagens Aéreas

---

Este projeto é um trabalho acadêmico desenvolvido para a disciplina de **Linguagens de Programação 2**. O objetivo é aplicar conceitos de Programação Orientada a Objetos (POO), design de interfaces gráficas (GUI) e arquitetura de software na construção de uma aplicação desktop funcional.



### Propósito do Projeto

---

O sistema simula um protótipo de portal de vendas de passagens aéreas. Seu propósito principal é permitir que um usuário realize o fluxo completo de uma compra, desde o cadastro e login no sistema até a busca de voos, seleção e simulação de reserva ou pagamento.

O foco não está apenas na funcionalidade final, mas na aplicação de conceitos como **herança** (telas que herdam de JFrame, JPanel), **encapsulamento** (classe Reserva), **gerenciamento de estado** (classes SessaoUsuario e DadosReservas) e **interação humano-computador (IHC)**.

### Funcionalidades Implementadas

---

O sistema atende aos principais requisitos funcionais (RF) de um sistema de e-commerce de viagens:

#### Autenticação de Usuário (RF03, RF04)

Telas de tela\_login e tela\_cadastro totalmente funcionais. O nome do usuário é persistido estaticamente na classe SessaoUsuario após o login e é corretamente exibido na tela de reservas.

Login



### Entrar na sua conta

Usuário

Senha

☐ Mostrar senha

**Entrar**

Ainda não tem uma conta? [Cadastre-se](#)

Cadastro de Usuário



### Crie sua conta

Nome

E-mail

Telefone

Senha

Confirmar senha

**Cadastrar**

[Voltar para o Login](#)

## Busca de Voos (RF01, RF02)

A TelaInicial permite a busca por origem e destino. O sistema valida buscas com a mesma cidade (RF01) e exibe resultados gerados aleatoriamente em TelaResultadosBusca.



Minhas Reservas Login

### Para onde você quer ir?

Origem:

Destino:

Datas (Ida e Volta):

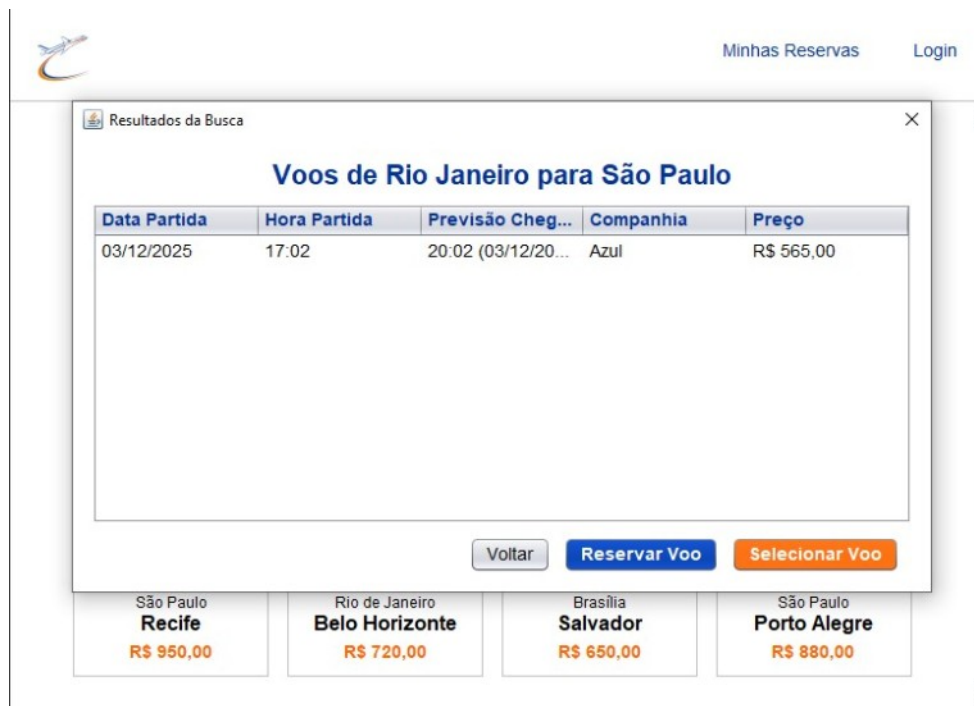
**Buscar Voos**

### Voos em Destaque

|                                          |                                                       |                                           |                                                |
|------------------------------------------|-------------------------------------------------------|-------------------------------------------|------------------------------------------------|
| São Paulo<br><b>Recife</b><br>R\$ 950,00 | Rio de Janeiro<br><b>Belo Horizonte</b><br>R\$ 720,00 | Brasília<br><b>Salvador</b><br>R\$ 650,00 | São Paulo<br><b>Porto Alegre</b><br>R\$ 880,00 |
|------------------------------------------|-------------------------------------------------------|-------------------------------------------|------------------------------------------------|

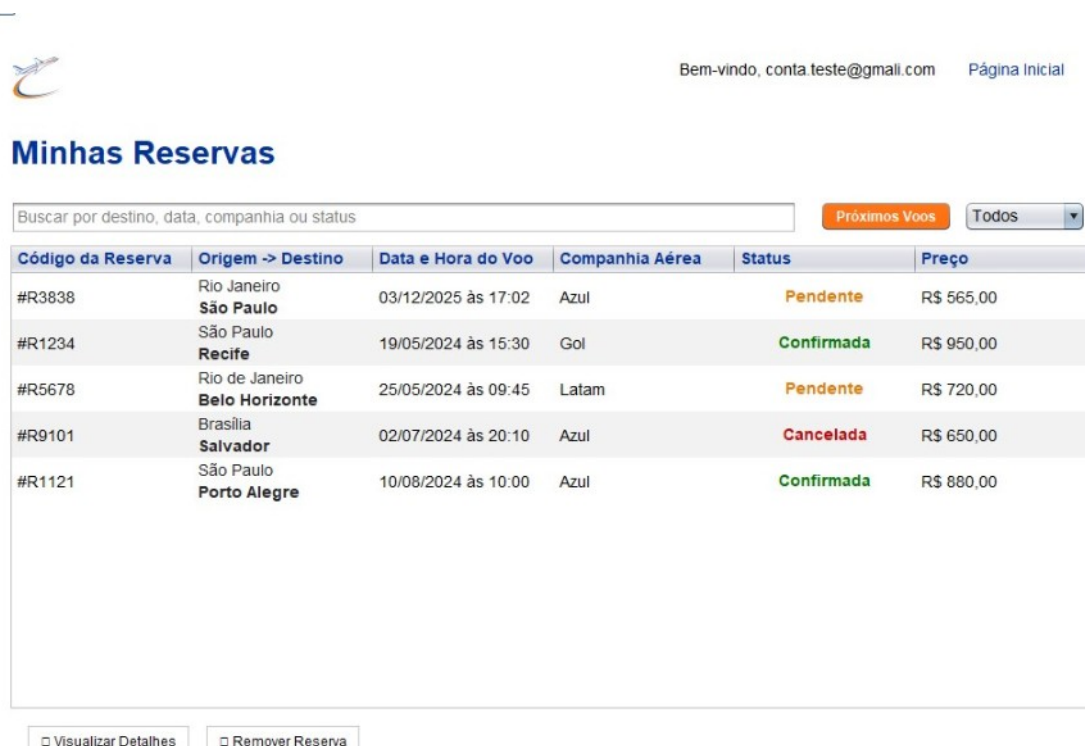
## Fluxo de Compra e Reserva (RF06, RF09)

Na tela de resultados, o usuário tem duas opções: "Selecionar Voo" (leva ao TelaPagamento) ou "Reservar Voo" (adiciona o voo à tela\_reserva com status "Pendente").



## Gerenciamento de Reservas (RF09, RF10)

A tela\_reserva lista todos os voos (mockados e recém-adicionados). O usuário pode clicar para ver detalhes (tela\_detalhes\_reserva) ou remover uma reserva.



## Detalhes e Emissão de Comprovante (RF08, RF10)

A tela de detalhes permite ao usuário editar dados (Nome, Documento), remover a reserva ou simular a geração de um comprovante em PDF.

The screenshot shows a web application interface with a modal window titled 'Detalhes da Reserva: #R3838'. The background shows a sidebar with 'Minhas Reservas' and a list of reservation codes (#R3838, #R1234, #R5678, #R9101, #R1121). The modal window is divided into three sections: 'Detalhes do Voo', 'Passageiro', and 'Pagamento'. The 'Detalhes do Voo' section shows flight status as 'Pendente', company as 'Azul (AZ 1084)', origin as 'Rio Janeiro (Aeroporto)', destination as 'São Paulo (Aeroporto)', departure as '03/12/2025 às 17:02 (Terminal 1, Portão A1)', and arrival as '20:02 (03/12/2025)'. The 'Passageiro' section shows the name 'Fulano', document as 'CPF Pendente', and seat as 'A-21'. The 'Pagamento' section shows a total value of 'R\$ 565,00', purchase date as '17/11/2025', and payment method as 'Pagamento Pendente'. At the bottom of the modal, there are buttons for 'Efetuar Pagamento', 'Remover Reserva', and 'Voltar'. The background interface also includes a search bar, a 'Página Inicial' link, and a table of reservations with prices ranging from 565,00 to 880,00.

| Detalhes do Voo |                                             |
|-----------------|---------------------------------------------|
| Status:         | Pendente                                    |
| Companhia:      | Azul (AZ 1084)                              |
| Origem:         | Rio Janeiro (Aeroporto)                     |
| Destino:        | São Paulo (Aeroporto)                       |
| Partida:        | 03/12/2025 às 17:02 (Terminal 1, Portão A1) |
| Chegada:        | 20:02 (03/12/2025)                          |

| Passageiro |              |
|------------|--------------|
| Nome:      | Fulano       |
| Documento: | CPF Pendente |
| Assento:   | A-21         |

| Pagamento       |                    |
|-----------------|--------------------|
| Valor Total:    | R\$ 565,00         |
| Data da Compra: | 17/11/2025         |
| Método:         | Pagamento Pendente |

## Tecnologias e Conceitos

- **Linguagem:** Java
- **Interface Gráfica (GUI):** Java Swing
- **Conceitos de POO:** Herança, Encapsulamento e Polimorfismo.
- **Design de Software:**
  - **Separação de Responsabilidades:** As telas (Views) são separadas da lógica de dados (Reserva, DadosReservas).
  - **Classes Utilitárias (Estáticas):** SessaoUsuario e DadosReservas atuam como serviços centralizados para gerenciar o estado da aplicação.
  - **Modelo de Dados (POJO):** A classe Reserva modela a entidade de negócio.

## Discussão: Escalabilidade

---

A arquitetura do projeto foi pensada de forma modular, o que facilita a **manutenibilidade**. Cada tela (JFrame, JPanel1) cuida de sua própria lógica de exibição, e os dados são centralizados, o que é um bom princípio de design.

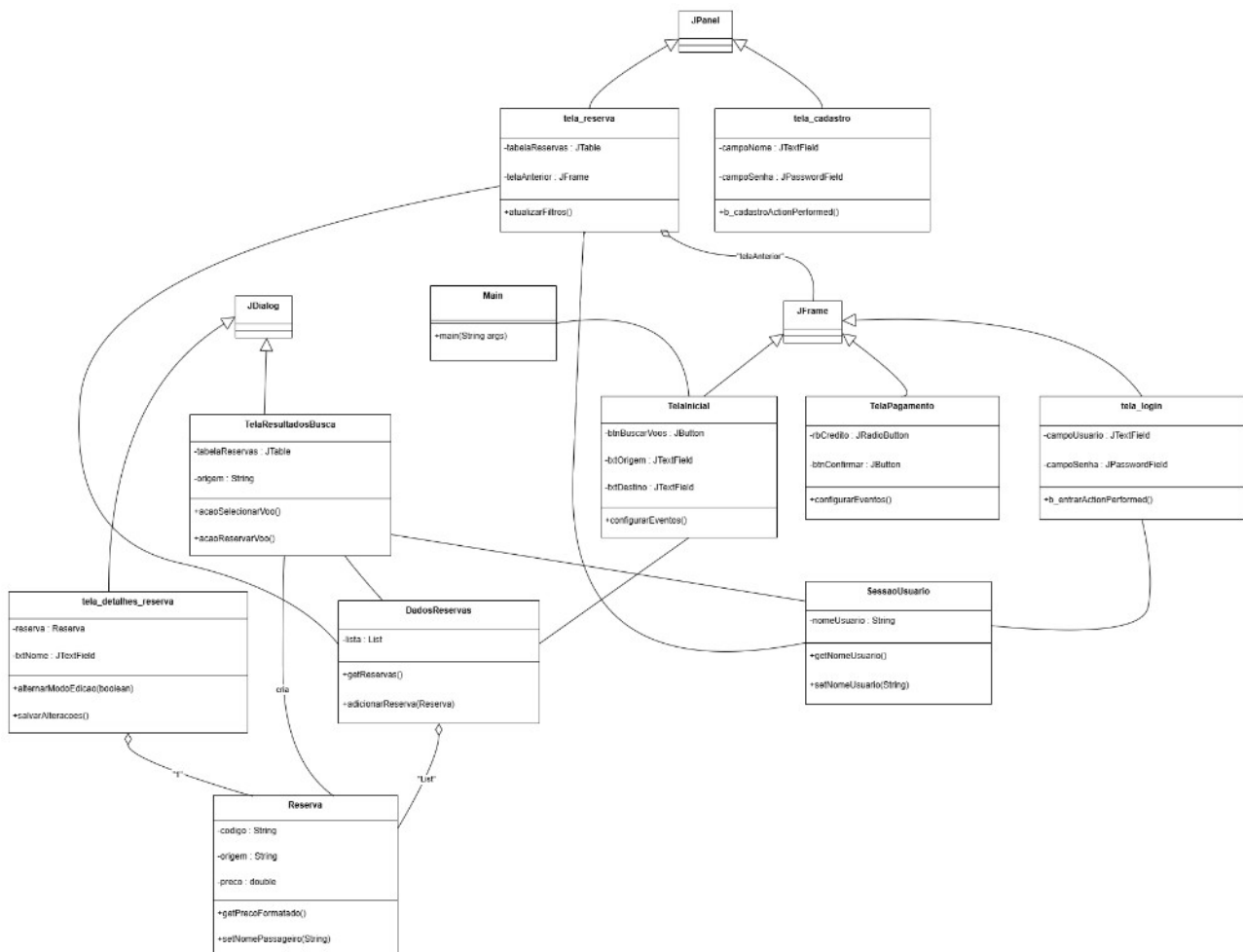
No entanto, a **escalabilidade** da aplicação em seu estado atual é **limitada** por dois fatores principais:

1. **Tecnologia (Desktop):** Sendo uma aplicação Java Swing, ela é monolítica e roda inteiramente na máquina do cliente. Ela não escala para múltiplos usuários simultâneos, como uma aplicação web faria.
2. **Persistência de Dados:** O uso de uma classe estática (DadosReservas) como "banco de dados" é o principal gargalo. Os dados são voláteis (perdidos ao fechar a app) e não podem ser compartilhados entre diferentes instâncias da aplicação.

Para que este projeto se tornasse escalável, o próximo passo seria refatorá-lo para uma **arquitetura Cliente-Servidor**:

- A aplicação Java Swing se tornaria puramente o **Cliente (Frontend)**.
- Toda a lógica de negócios (DadosReservas, SessaoUsuario) seria movida para um **Servidor (Backend)**, provavelmente uma API REST (feita em Spring Boot, por exemplo).
- O Backend se conectaria a um **Banco de Dados** real (como MySQL ou PostgreSQL) para persistir os dados.

## Diagrama de classes do sistema:



## Pontos de Melhoria

Embora o projeto seja funcional para seus objetivos educacionais, existem diversos pontos para melhoria e evolução:

- 1. Banco de Dados Real:** Substituir a classe `DadosReservas` por uma conexão JDBC a um SGBD (MySQL, PostgreSQL, etc.) para persistência real dos dados.
- 2. Autenticação Real:** Implementar um sistema de login que valide as credenciais com o banco de dados e utilize criptografia (hashing) para as senhas (RNFO3).
- 3. Validação de Campos:** Melhorar a validação dos formulários de cadastro e busca (ex: usar regex para e-mail, impedir buscas em datas passadas).
- 4. Componentes Avançados (IHC):** Substituir o campo de data (`JFormattedTextField`) por um componente de calendário (`JDatePicker`) para melhorar a usabilidade.

**5.Implementação de Requisitos Faltantes:** Adicionar as funcionalidades descritas no documento de requisitos que não foram implementadas, como a seleção de **número de passageiros** (RF01) e o método de pagamento via **PIX** (RF07).

**6.Tratamento de Erros:** Melhorar o tratamento de exceções (ex: falhas de conexão com o "banco", formatos de data inválidos) para fornecer feedback mais claro ao usuário.

---